Introduction to Software Systems
COMP 206

Max Kopinsky and Jacob Errington
School of Computer Science
McGill University

Lesson 2: File archives, permissions, and the Vim editor

Last time...

- The operating system and its role
- Context and importance of Linux
- Files and the way they're organized
- Paths
- Some practice

Plan for today

- Theory:
  - Archives.

Plan for today

- Theory:
  - Archives.
  - Permissions.

Plan for today

- Theory:
  - Archives.
  - Permissions.
  - Vim text editor.

Plan for today

- Theory:
  - Archives.
  - Permissions.
  - Vim text editor.
- Practice:
  - wget/tar
  - More terminal navigation
  - Vim text editor.

Resource Credit

- In past terms this has been a demo-only lecture...
- But I think having accompanying slides is important

- These slides are based on slides by prof Elsaadawy

Archives

- An archive is a collection of files combined into one file.
  - Being one file, archives are easier to manipulate (move, store, copy, backup, etc).
  - Archives are often compressed, so they require less space.

- TAR, GZIP, GUNZIP
  - The two most common archive tools used on Unix systems is tar and gzip (gunzip).
  - tar allows you to combine several files into a single file.
  - gzip allows you to compress a single file.
  - Combine AND compress: you need to use both tar and gzip.

Tar

- Allows the manipulation (creation, extraction) of archive files.
  - A file ending with the .tar extension is a tar archive file.
  - A file ending with the .tgz extension is a compressed (gzipped) tar archive file.
  - (Remember: file extensions are convention.)
- Switches:
  - -c : create a new tar archive
  - -x : extract from the tar archive
  - -f: specifies the archive file name.
  - -v: activates verbose mode, which means the tar command will output lots of information.
  - -z: automatically invoke g(un)zip for (de)compression

Tar (Cont.)

- Here are a few example of the tar command:
  - tar -cvf log.tar *.log
  - tar -zcvf log.tgz *.log
  - tar -xvf log.tar
  - tar -zxvf log.tgz

Tar (Cont.)

- Here are a few example of the tar command:
  - tar -cvf log.tar *.log
    - Create (verbosely) log.tar from all .log files in current directory
  - tar -zcvf log.tgz *.log
    - Create (verbosely) compressed log.tgz as above
  - tar -xvf log.tar
    - Extract (verbosely) all files in log.tar to ./log/
  - tar -zxvf log.tgz
    - Extract (verbosely) as above, with decompression
- You need to specify the files you want to archive. Use * if you want to archive all files in the working directory. (More on this syntax next week.)
- Use -C path/to/destination → extract the files into “path/to/destination” instead of the current directory.

Start Connecting to Mimi

- If you're on the right side of the room (your right, not mine)
- Start logging in to Mimi!

- To fix some issues from last time…
- You **must** either be on McGill WiFi or the VPN.
- Eduroam, the guest network, or your phone hotspot will **not** work.

- If you get an issue like "could not chdir to <home directory>: no permission" then you will need to contact IT to fix it.

- If you still cannot connect, try work with someone who can.

- If you're at center/left of room, be patient.

Permissions on the File Systems

- All files are owned by a user and a group.
  - Usually, this owner is the user that created the file.
- Permissions on files exists at three level:
  user, group and others.
- Three types of rights can be given:
  read, write and execute.
- Any combination of these rights must be given to these three levels.

View permissions of files

- ls –l / ls –l a.out

owner

group

```
melsaa2@teach-node-03:~$ ls -l
total 699
drwxr-xr-x  melsaa2 nogroup      6 Mar 21 11:33  A1
-rwxr-xr-x  melsaa2 nogroup  16704 Nov  6  2023  a.out
-rwxr-xr-x  melsaa2 nogroup   2777 Mar 12 11:25  checkfiles.sh
-rw-r--r--  melsaa2 nogroup   8285 Feb 15  2023  check.sql
-rwxr-xr-x  melsaa2 nogroup   1072 Mar 12 11:45  checks.sh
-rw-r--r--  melsaa2 nogroup   1754 Jan 30  2025  classexample.sql
-rw-r--r--  melsaa2 nogroup    352 Feb  6  2024  delNdrop.log
-rwxr--r--  melsaa2 nogroup    209 Feb  6  2024  delNdrop.sh
-rw-r--r--  melsaa2 nogroup 473920 Sep  3 21:59 'Ed-DP2UG-B1Eod09WaEoyb8BhUy1_xLu7tUie9LQUryufA?e=IMFvIY'
-rwxr-xr-x  melsaa2 nogroup    773 Mar 19 13:38  env.sh
-rwxr-xr-x  melsaa2 nogroup   2505 Mar 12 11:45  evalPGsubmissions.sh
-rwxr-xr-x  melsaa2 nogroup    452 Mar 12 11:34  genmasterpg.sh
-rwxr-xr-x  melsaa2 nogroup   1439 Mar 12 11:47  genreport.sh
```

links

size

Modification date

Permissions

Permissions (cont.)

d,rwx,rwx,rwx

- Permissions are displayed as a string of 10 characters
  - 1st : indicates the entry's type (file or directory).
  - 2nd : indicates if the owner has read access to the file.
  - 3rd : indicates if the owner has write access to the file.
  - 4th : indicates if the owner has execute access to the file.
  - 5th , 6th , 7th : indicates if the group owner has read, write or execute, respectively.
  - 8th , 9th, 10th : indicates if all other users have read, write or execute, respectively.

Example

- Given the permission “------rwx“ of a file I own, can I read the file?

Example

- Given the permission “------rwx“ of a file I own, can I read the file?

  - You will not be able to read the file.
  - People in the group will not be able to read the file.
  - Other people will be able to read the file.

Do permissions overlap?

- File permissions are evaluated in a specific order: owner, then group, then others.
- Example: Can user “Bob” of group “Student” read, write or execute the following files?

  - ----rwx--- Bob Student file1.txt

Do permissions overlap?

- File permissions are evaluated in a specific order: owner, then group, then others.
- Example: Can user “Bob” of group “Student” read, write or execute the following files?
  - ----rwx--- Bob Student file1.txt
- No!

CHMOD – change mode

- The chmod command is used to change permissions:
- Who:
  - u : The user who owns the file (this means “you.”)
  - g : The group the file belongs to.
  - o : The other users
  - a : all of the above (an abbreviation for ugo)
- Permission:
  - r : Permission to read the file (listing the entries of a directory).
  - w : Permission to write (or delete) the file (create new files in the directory).
  - x : Permission to execute the file ("traverse" for directories).

CHMOD – change mode (Cont.)

- Change to:
  - = (sets)
  - + (add)
  - - (remove)
- The syntax of the command is as follows:

```
chmod who=permission file(s)
```

CHMOD – change mode (Examples)

- Here are a few examples of the chmod command:
  - chmod +x foo

  -> adds (+) the execute permission for each of the three classes of users (owner, group members, and anybody else)
  - chmod g-w foo

  -> removes (-) the write permission from group members
  - chmod u+r foo

  -> adds the read permission for the file's owner
  - -chmod a=rw,u+x foo

  -> sets permissions for all classes of users to read+write, then additionally adds the execute permission only for the file's owner.

Binary settings

- Bit setting:
  - rwx rwx rwx → In symbolic form
  - 111 000 111 → In bit form (1=on, 0=off)
  - 707 → in base 8 version of bits.

    (Why base 8? 3 binary bits = 2^3 = 8.

    Much easier to read for humans!)

000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7
rwx	

Binary settings

- Bit setting:
  - rwx rwx rwx → In symbolic form
  - 111 000 111 → In bit form (1=on, 0=off)
  - 707 → in base 8 version of bits.

000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7
rwx	

- chmod 707 foo → rwx for owner and other, but not for group

Binary settings

- Bit setting:
  - rwx rwx rwx → In symbolic form
  - 111 000 111 → In bit form (1=on, 0=off)
  - 707 → in base 8 version of bits.
- chmod 707 foo → rwx for owner and other, but not for group
- Exercise: what are the permissions that are assigned by the following command?

  chmod 610 foo

Binary settings

- Exercise: what are the permissions that are assigned by the following command?

chmod 610 foo

rw-	--x	---
110	001	000
6	1	0

000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7
rwx	

Practice

- You may want to create a cs206 folder on Mimi
  - (if you haven't already)

- Download today's first exercise *directly in your terminal!* Cool!
  - $ wget cs.mcgill.ca/~mkopin/cs206/treasure-hunt.tgz
  - ~ is the same character as last week
- Yep, that's a compressed archive. Extract it…
  - $ tar –xzf treasure-hunt.tgz

- Then enter the hunt, read the instructions, and go!
  - $ cd treasure-hunt
  - $ cat README.md

The vim Text Editor

- Command line text editor allows you to create/edit files at the command line.
- One of the original text editors available on Unix. It's easy to learn the basics. However, it's very powerful and available on every Unix machine.
- Difficult to master – I've used it almost daily for nearly 10 years and still learn a new capability of vim every couple months!
  - This is not a bad thing, because the basics get you **very** far
  - But you can keep boosting your productivity incrementally

Vim's modes

- "Modal editor": keys do different things in different modes
  - Vim designed to be "mouseless" – keyboard only
- Normal Mode:
  - Default mode when you open vim.
  - Used to navigate, copy and paste text.
  - can use special one letter commands.
  - can use arrow keys, but really slows you down. This is a crutch.
  - Most time is spent in normal mode
- Insert Mode:
  - to edit your text.
  - can press any keyboard characters.
  - To enter the mode for inserting text, press i
  - To exit insert mode and return to normal mode, press ESC.
- Command Mode:
  - issue commands like Save, Quit, Search, etc.

Vim “motion”

- Can easily move around file in normal mode **without mouse**
- Takes a bit of getting used to though!

  - h: move cursor left
  - j: move cursor down
  - k: move cursor up
  - l: move cursor right (lowercase L)
  - (If you use an unusual keyboard layout like Dvorak, search for how to use it with vim.)

  - Type number **before** any command to repeat

- gg: move to start of file
- G: move to end of file
- 0: move to start of line
- $: move to end of line

- w: move to start of next word
- e: move to end of current word
- b: move to previous beginning of word
- f<char>: move to next instance of <char> on the current line

Important vim Commands

To get help for a command
:help command

- To get into insert mode: Any of the following: i, I, a, A, o, O

  - i – insert before the cursor
    I – insert at start of line before first non-blank

  - a – insert after the cursor
    A – insert at the end of the line

  - o – begin a new line below the cursor and insert there
    O – same but above cursor

© 2023

Important vim Commands

To get help for a command
:help command

- Normal Model: press ESC to get into normal mode.

- To delete: dd, D, x, r

  dd – delete a line | D – delete the rest of the line
  x – delete a character | r – replace a character

- To search: / (forward), ? (backwards)
  Once you press enter, press n and N to get next/previous result

- To copy (yank) and paste a line:
  yy or Y (equivalent) – yank
  p - paste before cursor | P – paste after cursor

© 2023

Important vim Commands

To get help for a command
:help command

- Command mode: press : to get into command mode
  w, q, wq, q!, line number, e filename

:w – save file

:q – quit current window

:wq save and quit

:q! quit and discard changes

:number – go to a specific line number (For example, :14)

:%s/<search pattern>/<replacement pattern> – find and replace

:e filename Edit a file

© 2023

Practice your vim skills

- Command line, in mimi: $ vimtutor
- A website to practice vim interactively: https://openvim.com/
- Do a quiz on vim commands once you have practiced quite a bit: http://www.vimgenius.com/start